

2025 Portfolio.

게임 개발자 홍길동 포트폴리오

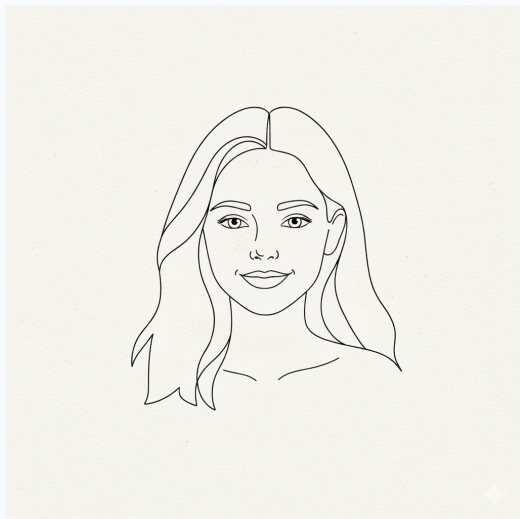
디테일한 개발자 홍길동입니다!

CONTACT

gildong@gmail.com

010-1234-1234

About Me



홍길동

연락처 010-1234-1234
이메일 gildong@gmail.com
생년월일 1998.12.17
주소 서울특별시 구로구

EXPERIENCE

- 2025.07 GIGDC2025 제작부문 참가
- 2024.12 – 2025.06 [원티드 포텐업] 데이터·AI & 게임 클라이언트 협업 트랙 1기 수료
최종 협업 프로젝트 최우수상 수상
- 2024.05 – 2024.10 공간 디자인 회사 디자인팀 근무
팝업 스토어 설계 | 디자인 및 설계 도면 작성

LINK

- 깃허브 <https://github.com/gildong>
노션 <https://buly.kr/ddddddd5IH>
블로그 <https://velog.io/@gildong/>

STUDY

- CS:APP
OSTEP

SKILLS



Unreal Engine



C / C++



SketchUp



Photoshop

Projects



01

Recreating Presona5

페르소나 5: UI

JRPG

- UE5 CommonUI
- 게임 연출
- 미니맵 UI



02

원티드 포텐업 최우수상

올라락! 깨구락!

수직 플랫폼 게임

- 게임 UI/UX 개발
- 게임 내 오브젝트
- 게임 장애물 구현
- 음성 퀴즈 (AI 연동)



03

Recreating World of Tanks

Local Tanks

밀리터리 TPS

- 미니맵 UI
- 목적지 인디케이터 UI
- BehaviorTree를 활용한 AI 탱크 개발



04

Recreating Left 4 Dead 2

Right 4 Dead 2

서바이벌 FPS

- 무기 구조 설계
- 무기 슬롯 시스템
- 무기별 공격 로직
- 플레이어 / 관련 UI



05

Recreating Party Animals

Party Animals

캐주얼 액션 대전

- Steam 연동 로비
- <블랙홀> 맵 로직
- 중력 컴포넌트
- 관전자 모드 구현

01 페르소나 5: UI

[시연영상](#)

[깃허브 링크](#)



▶ 프로젝트 개요

페르소나5의 시각적 연출과 UI 시스템을 분석해,
언리얼 엔진5의 **CommonUI로 재현한** 프로젝트입니다.

태그 기반 UI 스택 구조, 전역 ESC 입력 처리,
AnimNotify를 통한 연출 제어, 미니맵 시스템 등을
설계하고 구현했습니다.

▶ 개발 진행 기간

2025.09 – 2025.10

▶ 작업 인원

1인

▶ 구현 내용

- Common UI 스택 구조, 입력 매핑 설계
- 애니메이션 및 상호작용 연출
- 인게임 미니맵 UI 시스템

▶ 사용한 기술

Unreal Engine 5, C++, Common UI

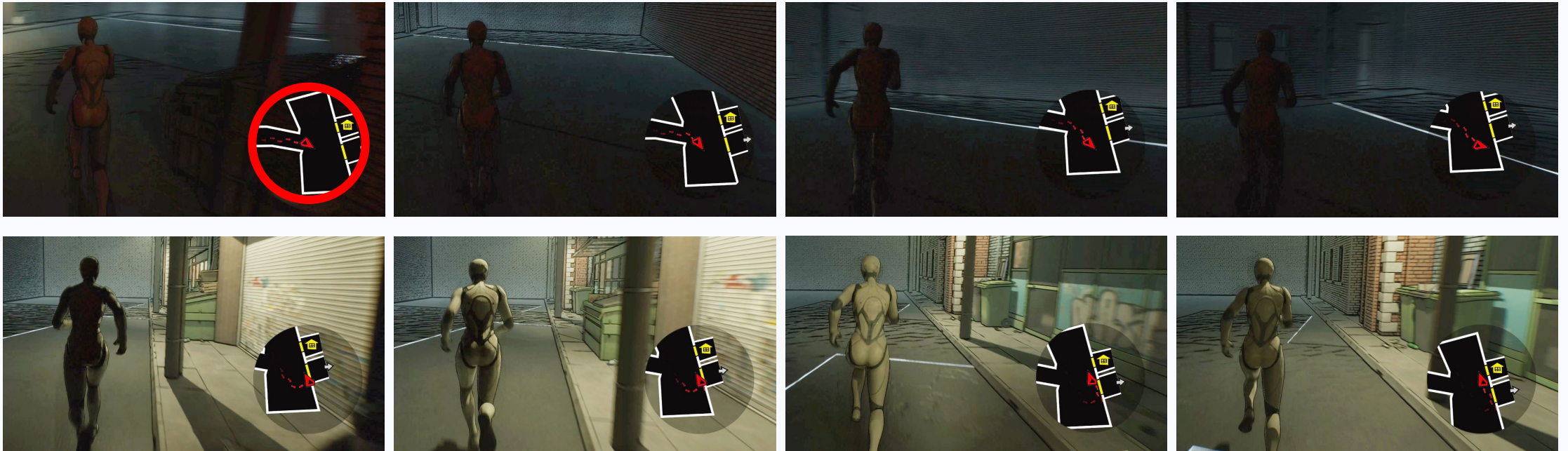
페르소나 5: UI

미니맵 UI

[정규화 및 방향 스크롤링 소스코드](#)[발자국 스폰 로직 소스코드](#)[PlayableBound 소스코드](#)

맵 범위 보정을 위한 **BoundingBox**를 배치한 후 플레이어의 월드 좌표를 UV 공간으로 변환, 플레이어를 중앙으로 고정하고
맵과 발자국이 반대 방향으로 스크롤되도록 설계했습니다.

발자국은 **외적**을 통해 왼발, 오른발 교차 보행 로직을 설계하였고, Material Instance를 **오브젝트 풀링**으로 관리했습니다.



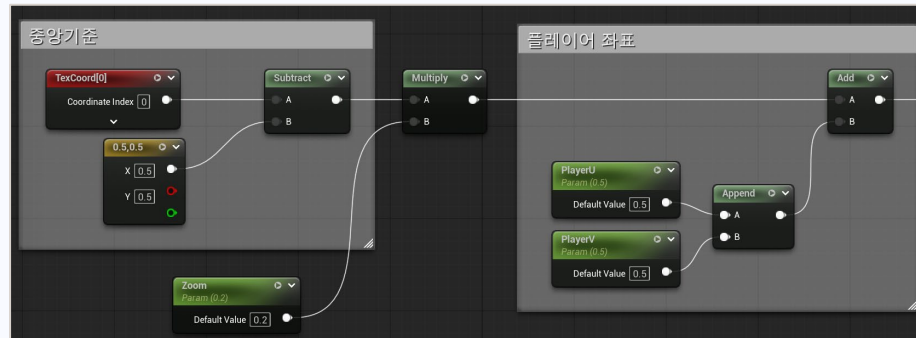
플레이 환경 내 캐릭터의 이동과 함께 반응하는 미니맵 연출 시퀀스

페르소나 5: UI

미니맵 UI : 구현 설명

[정규화 및 방향 스크롤링 소스코드](#)[발자국 스폰 로직 소스코드](#)[PlayableBound 소스코드](#)

▶ 플레이어 중심 UV 리맵핑



중심 고정형 스크롤 및 줌

```
void APlayableBound::GetWorldXYMinMax(FVector2D& OutMin, FVector2D& OutMax) const
{
    const FVector Origin = GetActorLocation();
    const FVector Extend = Bounds->GetScaledBoxExtent();
    OutMin = FVector2D(InX: Origin.X - Extend.X, InY: Origin.Y - Extend.Y);
    OutMax = FVector2D(InX: Origin.X + Extend.X, InY: Origin.Y + Extend.Y);
}

PlayableBound->GetWorldXYMinMax([&] MapMin, [&] MapMax);
```

맵 범위 보정을 위한 PlayableBound 배치

▶ 발자국 오버레이 시스템

```
FVector Right = FVector::CrossProduct(A: FVector::UpVector, B: Dir).GetSafeNormal();
float SideSign = (NextFootIndex % 2 == 0) ? -1.f : +1.f; // 짝수=왼, 홀수=오른

FVector SpawnWorld = Pawn->GetActorLocation()
    - Dir * BackOffset
    + Right * SideSign * SideOffset;
```

보폭 방향 계산 시 외적을 통해 진행 방향에 직교하는
화면 우측 벡터를 얻고, 짝수/홀수 스텝에 ±를 적용해
좌/우 발자국 위치를 산출

```
const float Speed = Pawn->GetVelocity().Size2D();
if (Speed > MoveSpeedThreshold)
{
    AccumDist += static_cast<double>(Speed) * static_cast<double>(DeltaSeconds);

    while (AccumDist >= StepDistanceWorld)
    {
        AccumDist -= StepDistanceWorld;
        SpawnFoot();
    }
}
```

거리 누적 로직으로 이동속도와 무관한 일정 간격 발자국 생성

페르소나 5: UI

[MainMenuPawn 소스코드](#)

메인 메뉴 진입 연출

착지 애니메이션의 각 구간(StartPose → LandLoop → Stand)을 **AnimNotify**로 제어해 착지 타이밍에 맞춰 카메라 전환과 UI 표시를 자연스럽게 연결했습니다.

▶ 연출 흐름



```
Anim->Montage_JumpToSection(FName("StartPose"), LandToStandMontage);  
Anim->Montage_Pause(LandToStandMontage);
```

캐릭터가 바닥 착지 전까지
Land 포즈를 유지



```
Anim->Montage_Resume(LandToStandMontage);
```

캐릭터가 바닥에 착지 시
몽타주 다시 재생



```
PC->SwitchCameraWithShake(PressCam, BlendTime: 0.2f);  
PC->EnsureRootCreated(TAG_UI_Screen_PressAnyKey);
```

캐릭터가 일어나면서 **카메라 전환**
RootWidget 생성 후 **UI**를 스택에 **Push**

페르소나 5: UI

언리얼엔진 Common UI 시스템

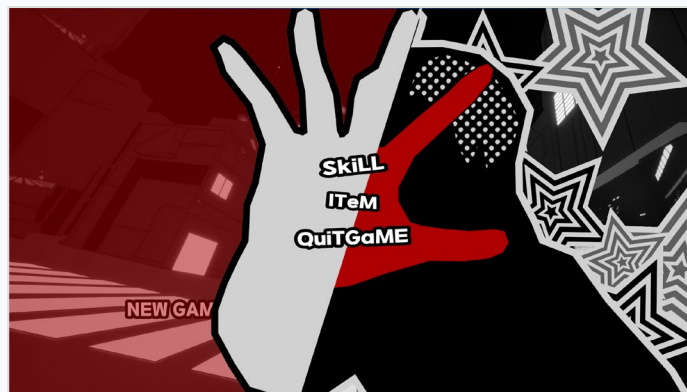
[GameplayTag 소스코드](#)[Push By Tag 로직 소스코드](#)

RootWidget을 중심으로 **Main·Overlay·Modal·HUD** 4중 스택을 설계하고, **GameplayTag** 기반으로 UI 화면을 Push/Pop 으로 관리했습니다. 모든 UI의 공통 부모(UBaseWidget)에 **전역 ESC 입력을 상주시켜**, 어떤 화면에서도 일관된 메뉴 토글이 가능하도록 구현했습니다.

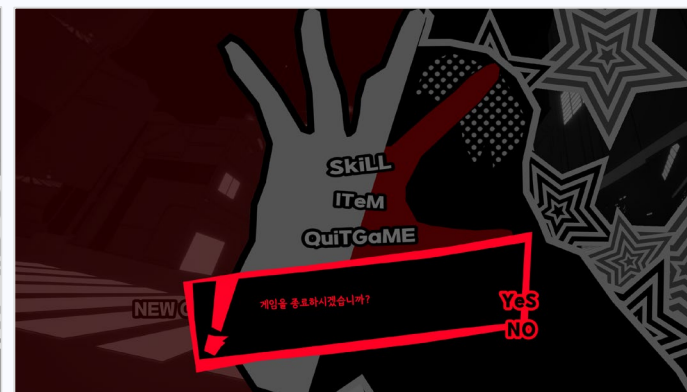
▶ 스택 위계



Main Stack



Overlay Stack



Modal Stack

HUD → Main → Overlay → Modal 4중 스택으로 관리
위로 갈수록 입력을 우선 소모하고, 아래 레이어 입력 비활성화

페르소나 5: UI

엔리얼엔진 Common UI 시스템 : 구현 설명

[GameplayTag 소스코드](#)[Push By Tag 로직 소스코드](#)

▶ 스택에 UI Push

```

UCommonActivatableWidget* URootWidget::PushByTag(const FGameplayTag ScreenTag)
{
    if (!ScreenTag.IsValid()) return nullptr;
    UE_LOG(LogTemp, Warning, TEXT("InFormat: %s"), *ScreenTag.ToString());

    // 태그로 클래스 찾기
    TSubclassOf<UBaseWidget> ScreenClass = ScreenMap.FindRef(ScreenTag);
    if (!ScreenClass) return nullptr;

    UBaseWidget* Base = ScreenClass.GetDefaultObject();
    FGameplayTag Layer = Base->GetLayerTag();
    UCommonActivatableWidgetStack* Target = ChooseStackByLayer(Layer);

    // 현재 활성화된 위젯이 같은 클래스면 그냥 반환
    if (UCommonActivatableWidget* Current = Target->GetActiveWidget())
    {
        if (Current->GetClass() == ScreenClass)
        {
            return nullptr;
        }
    }

    // 스택에 추가후 포인터 반환
    return Target ? Target->AddWidget(*ScreenClass) : nullptr;
}

```

Tag 기반으로 UI Push 요청

```

UCommonActivatableWidgetStack* URootWidget::ChooseStackByLayer(const FGameplayTag& LayerTag)
{
    if (LayerTag == TAG_UI_Layer_Main) return MainStack;
    if (LayerTag == TAG_UI_Layer_Overlay) return OverlayStack;
    if (LayerTag == TAG_UI_Layer_Modal) return ModalStack;
    if (LayerTag == TAG_UI_Layer_HUD) return HUDStack;
    return nullptr;
}

```

Tag 기반으로 스택 구분

▶ 전역 ESC 처리

```

void UBaseWidget::NativeOnInitialized()
{
    Super::NativeOnInitialized();

    if (IA_Escape)
    {
        RegisterUIActionBinding(FBindUIActionArgs(
            IA_Escape, bShouldDisplayInActionBar: false,
            InOnExecuteAction: FSimpleDelegate::CreateUObject(this, &UBaseWidget::OnToggleMenu)));
    }
}

```

BaseWidget에서ESC Input 바인딩

ENHANCED INPUT	ENHANCED INPUT
Player: BP_InGamePlayerController_C_0	Player: BP_ThirdPersonPlayerController_C_0
Input Mode: (GameplayTags=((TagName="Enhanced")))	Input Mode: (GameplayTags=((TagName="Enhanced")))
Context: IMC_Global Priority: 200 Tracking Mode: false	Context: IMC_Global Priority: 200 Tracking Mode: false
Action: IA_Escape - None - 0.000s (false)	Action: IA_Escape - None - 0.000s (false)
Escape -	Escape -
Context: IMC_Game Priority: 0 Tracking Mode: false	Context: IMC_UI Priority: 10 Tracking Mode: false

현재 포커스/루트 UI에 상관없이 UBaseWidget 바인딩이 콜백

02 올라락! 깨구락!

시연영상

깃허브 링크

멀티플레이

AI 협업



▶ 프로젝트 개요

플레이어가 직접 맵을 제작(에디팅)하고 플레이할 수 있는
수직 플랫폼 게임입니다. WebSocket 기반 AI 서버와
음성 퀴즈 연동, 객체 풀링 기반 UI/장애물 최적화 등
기술적인 완성도를 높이는 데 집중했습니다.

▶ 개발 진행 기간

2025.04 – 2025.07

▶ 작업 인원

AI 3인 + 게임 클라이언트 3인

▶ 담당 역할

- 게임 UI/UX 개발
- 게임 내 오브젝트 및 장애물 구현
- 음성 퀴즈 시스템 (WebSocket 기반 AI 연동)

▶ 사용한 기술

Unreal Engine 5, C++, WebSocket

올라락! 깨구락!

[친구목록 UI 풀링 소스코드](#)

UI 오브젝트 풀링

친구 목록 및 방 목록 UI는 초기에 동적 생성 및 제거를 반복하며 매 프레임 UI를 갱신했으나, 목록이 많아질수록 **GC 부하**가 발생할 수 있음을 인지했습니다. 이를 해결하고자 **반복 생성되는 친구 목록/방 목록 UI를 미리 생성해 풀에 저장**하고, UI 갱신 시 **SetVisibility**로 표시 여부만 제어하는 구조로 설계를 변경했습니다.

구현 방식

1 초기 풀 생성 단계

```

ScrollBox_Friend->ClearChildren();
FriendsItemPool.Empty();
for (int32 i = 0; i < MaxFriendCount; i++)
{
    UFriendsItemWidget* FriendsItemWidget = CreateWidget<UFriendsItemWidget>(OwningObject: this, FriendsItemWidget);
    FriendsItemWidget->SetVisibility(ESlateVisibility::Collapsed);

    FriendsItemPool.Add(FriendsItemWidget);
    ScrollBox_Friend->AddChild(FriendsItemWidget);
}

```

Init() 함수 안에서 미리 위젯을 생성 후 배열(풀)에 저장

2 갱신 시 재활용

```

UE_LOG(LogTemp, Display, TEXT(InFormat: "FriendsList::UpdateFriendList"));
for (int i = 0; i < FriendsItemPool.Num(); ++i)
{
    if (i < FriendDataArray.Num())
    {
        FriendsItemPool[i]->SetFriendInfo(FriendDataArray[i]); // 데이터 바인딩
        FriendsItemPool[i]->SetVisibility(ESlateVisibility::Visible);
    }
    else
    {
        FriendsItemPool[i]->SetVisibility(ESlateVisibility::Collapsed);
    }
}

```

새로 만들지 않고 설정만 전환



올라락! 깨구락!

[Prop 설계 소스코드](#)

장애물 설계

게임 내 다양한 오브젝트 (장애물, 장식물, 건축물 등)의 **공통 기반 클래스인 ABaseProp**을 설계해 **상속 받는 구조**로 개발 협업의 안정성과 기능 확장성을 확보했습니다. 이를 바탕으로 **다양한 장애물을 구현**해 동작을 일관되게 제어했습니다.

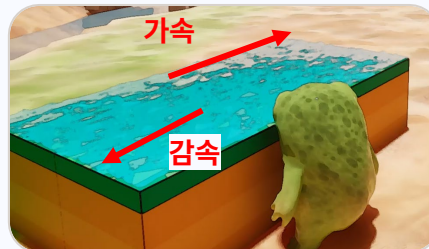
▶ 장애물 예시 | AObstacleProp

1 LaunchCharacter



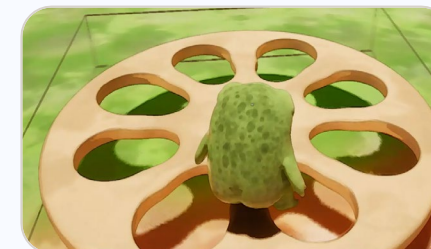
각 장애물에서 지정한
LaunchVelocity 방향으로 캐릭터를 튕겨냄

2 AddWorldOffset



캐릭터 진행 방향으로
가속 / 감속

3 AddLocalRotation



바닥이 회전되며
플레이어 진로 방해

올라락! 깨구락!

[장애물 동기화 소스코드](#)

장애물 멀티 플레이 동기화 | LaunchCharacter()

기존에는 서버가 충돌을 감지하고 튕김 반응을 적용했지만, 클라이언트-서버 간 위치 불일치로 충돌 감지 오류가 발생했습니다. 이를 해결하기 위해 클라이언트가 먼저 튕김 반응을 로컬에서 적용한 뒤, 서버에 요청하여 서버 권한으로 반응을 확정하고 Multicast로 동기화하는 방식으로 변경했습니다.

구현 방식

1 장애물에서 충돌 감지 → 서버에 요청

```
LaunchVelocity = Direction.GetSafeNormal() * Force;
if (Character->IsLocallyControlled())
{
    Character->StopCrouch();
    Character->LaunchCharacter(LaunchVelocity, bXYOverride, bZOverride);
    Character->ServerRPC_Launch(LaunchVelocity); // → 서버 요청 (위치이동!)
    ForceNetUpdate(); // 다음 틱에서 무조건 이 액터의 상태를 복제하도록
}
```

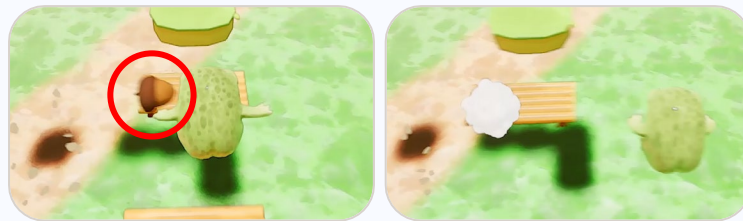
2 서버 기준으로 Launch 확정 + Multicast

```
if (HasAuthority())
{
    GetCharacterMovement()->Launch(LaunchVelocity);
}

MulticastRPC_Launch(LaunchVelocity);
ForceNetUpdate();
```

3 다른 클라이언트에서 동기화

```
if (GetLocalRole() == ROLE_SimulatedProxy)
{
    GetCharacterMovement()->Launch(LaunchVelocity);
    ActivateRecentlyLaunchedFlag();
}
```



튕겨져 나가는 모습

올라락! 깨구락!

[WebSocket 기반 음성 퀴즈 장애물 소스코드](#)

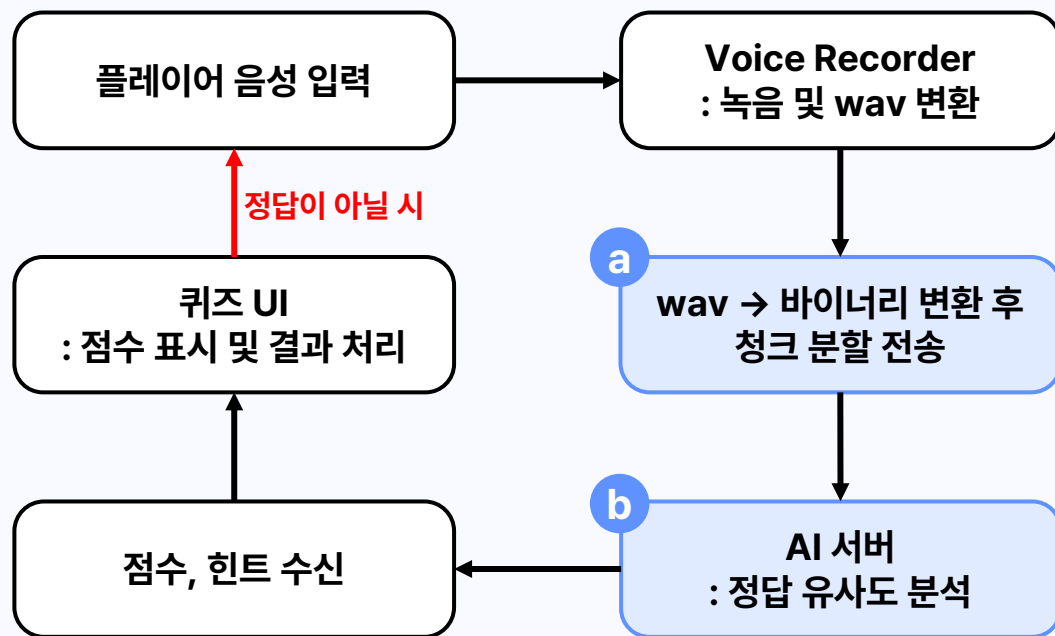
WebSocket 기반 음성 퀴즈 | AI 협업

실시간 녹음된 음성을 wav로 저장한 뒤, 30KB로 분할된 **청크 데이터를 WebSocket으로 전송**,

AI 서버로부터 유사도 분석 결과를 받아 퀴즈 성공 여부를 판단합니다.

순서 보장, 패딩 처리 등을 고려해 **멀티플레이 환경에서도 안정적으로 동작하는 구조를 구현했습니다.**

▶ 구조 다이어그램



a 청크 분할

```
// 2. 쪼개서 여러 패킷으로 전송 (30720바이트 단위로)  
// 현재 인덱스 기준으로 청크를 생성  
// WAV 전체에서 30720(ChunkSize)만큼 데이터를 꺼내고, 마지막 청크는 30720보다 작을 수 있으니까 Min()으로  
int32 CopySize = FMath::Min(A.ChunkSize, B.TotalWavSize - (CurrentSendIndex * ChunkSize));  
FWavRequestMessage ReqMessage;
```

b AI 서버로부터 결과 받기

```
// 정답 여부 판단  
Similarity = RespMessage.WavResponseMessage.Similarity;  
bSuccess = RespMessage.WavResponseMessage.bSuccess;
```

올라락! 깨구락!

[PlayerStar 동적 생성 소스코드](#)

트러블 슈팅

▶ Prop | PlayerStart 동적 생성 스폰 타이밍 문제 해결

상황

맵에 배치된 PlayerStart가 아닌, 플레이어가 직접 배치 가능한 StartProp을 제작했으나 언리얼엔진이 플레이어 스폰을 시도할 때 **아직 StartProp이 월드에 로드되지 않아 ChoosePlayerStart() 호출 시 올바른 위치를 찾지 못해** 기본 랜덤 PlayerStart가 선택되었습니다.

해결 방법

SpawnDefaultPawnFor 단계에서 **맵이 준비되지 않았다면 해당 플레이어를 TArray에 보관하고 스폰을 보류했습니다.** 이후 맵 로딩 완료되면 TArray 목록을 순회하며 **ChoosePlayerStart를 다시 호출해 각 플레이어를 정상적으로 스폰**하는 구조로 변경해 정확한 위치에 캐릭터 스폰을 성공했습니다.

배운 점

캐릭터 스폰 과정은 여러 단계의 내부 호출이 얹혀 있어, 문제 해결에는 **체계적인 로그 기반 디버깅이 필수적**임을 배웠습니다. PlayerStart 없이도 원하는 순간과 위치에서 스폰을 제어할 수 있는 확장 가능성을 확인했습니다.

```
APawn* AInGameMode::SpawnDefaultPawnFor_Implementation(AController*
{
    if (!bIsMapReady)
    {
        // 로그만 남기고 스폰 보류
        if (!PendingPlayers.Contains(NewPlayer))
        {
            PendingPlayers.Add(NewPlayer);
        }
        FFastLogger::LogConsole(Format::TEXT("SpawnDefaultPawnFor_Im
    }

    return nullptr;
}
```

스폰 보류



```
void AInGameMode::OnMapLoaded()
{
    bIsMapReady = true;

    FFastLogger::LogConsole(Format::TEXT("AInGameMode::OnMapLoaded"));
    for (AController* Controller : PendingPlayers)
    {
        AActor* Actor = ChoosePlayerStart(Controller);
        FFastLogger::LogConsole(Format::TEXT("AInGameMode::OnMapLoaded :
        RestartPlayerAtPlayerStart(Controller, Actor); // 다시 스폰 시도
    }
}
```

로딩 완료 후 스폰 시도

올라락! 깨구락!

[고정된 복사본 생성 소스코드](#)

트러블 슈팅

▶ 장애물 | Overlap 이벤트 이터레이터 무효화 문제 해결

상황

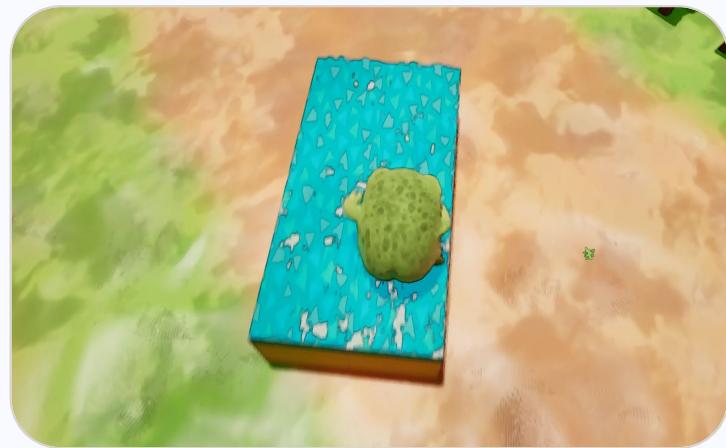
컨베이어 벨트가 겹친 개체를 매 틱 이동시키는 동안 OverlappingFrogs를 순회할 때, 이동으로 장애물을 벗어나는 순간 같은 틱에 **Begin/End Overlap** 이벤트가 발생해 순회 도중 수정되었습니다. TSet은 원소 삽입/해제 시 해시 버킷 재배치가 일어나 **이터레이터 무효화**가 발생, 일부 개구리가 누락되거나 중복 처리되는 불안정 현상이 나타났습니다.

해결 방법

틱 시작시 **고정된 복사본을 만들었고 처리 루프는 복사본만 순회하게 변경**하여 이번 프레임 동안의 처리 대상이 고정되도록 만들었습니다. 또한 순회 시에는 IsValid로 파괴된 개체를 건너뛰어 안정성을 확보하였습니다.

배운 점

TSet이나 TMap처럼 내부 재배치가 가능한 컨테이너는 이벤트 타이밍과 상호작용하며 버그를 유발할 수 있으므로, 실시간 시스템에서 순회와 수정을 분리하는 것이 중요하다는 점을 배웠습니다.



컨베이어 벨트 장애물

```
void AConveyorBeltProp::ConveyorMove()
{
    CopyOverlapFrogs = OverlappingFrogs;
    // 유효한 Frog들만 처리
    for (AFrog* Frog : CopyOverlapFrogs)
    {
        if (!IsValid(Frog))
            continue;

        FVector DeltaPos = BeltSpeed * GetWorld()->DeltaTimeSeconds * BeltDir;

        Frog->AddActorWorldOffset(DeltaPos, bSweep: true, OutSweepHitResult: nullptr,
    }
}
```

복사본 처리

03 Local Tanks

[시연영상](#)

[깃허브 링크](#)



▶ 프로젝트 개요

다채로운 전술 성향을 가진 **AI 탱크**와 전투하는 게임을 개발했습니다. **Behavior Tree**를 기반으로 **AI의 행동을 설계**하고, **직관적 UI**를 통해 전략적 플레이를 지원했습니다.

▶ 개발 진행 기간

2025.01 – 02

2025.08에 코드 개선

▶ 작업 인원

3인

▶ 담당 역할

- 미니맵 UI
- 점령지 Off-Screen Indicator UI
- BehaviorTree를 활용한 AI 탱크 개발
- 데미지 시스템 설계

▶ 사용한 기술

Unreal Engine 5, Blueprint, C++, Behavior Tree

Local Tanks

[미니맵 UI 좌표 변환 소스코드](#)

미니맵 UI

맵 지형이 고정되어 있다는 점을 고려해 **지형은 시작 시 SceneCaptureComponent로 1회 촬영한 RenderTarget** 텍스처를 사용했습니다. 또한 플레이어 · 적 · 목적지는 **월드 → 미니맵UI 좌표 변환을 통해 위치와 회전만 실시간 갱신하는 형태**로 설계하여 **런타임 캡처 비용을 제거한 미니맵**을 구현했습니다. 추후 맵이 확대/축소 되더라도 **마커는 월드 좌표의 정규화 비율(UV)**을 기준으로 배치되어 **지형 대비 상대 위치가 그대로 유지**됩니다.

1 월드 좌표 → 정규화된 UV 좌표

```
// 카메라의 Right/Up 벡터(평면 투영)로 성분 투영 → 카메라 화면 좌/우, 상/하로 분해
const FVector2D Right2D(Capture->GetRightVector().X, Capture->GetRightVector().Y); // 화면 가로축
const FVector2D Up2D(Capture->GetUpVector().X, Capture->GetUpVector().Y); // 화면 세로축

const float Xr = FVector2D::DotProduct(A: Delta, B: Right2D); // [-HalfW, +HalfW]
const float Yu = FVector2D::DotProduct(A: Delta, B: Up2D); // [-HalfH, +HalfH]

// 정규화 [0,1]
float U = (Xr + HalfW) / (OrthoWidth); // 가로 방향
float V = (Yu + HalfH) / (OrthoHeight); // 세로 방향
```

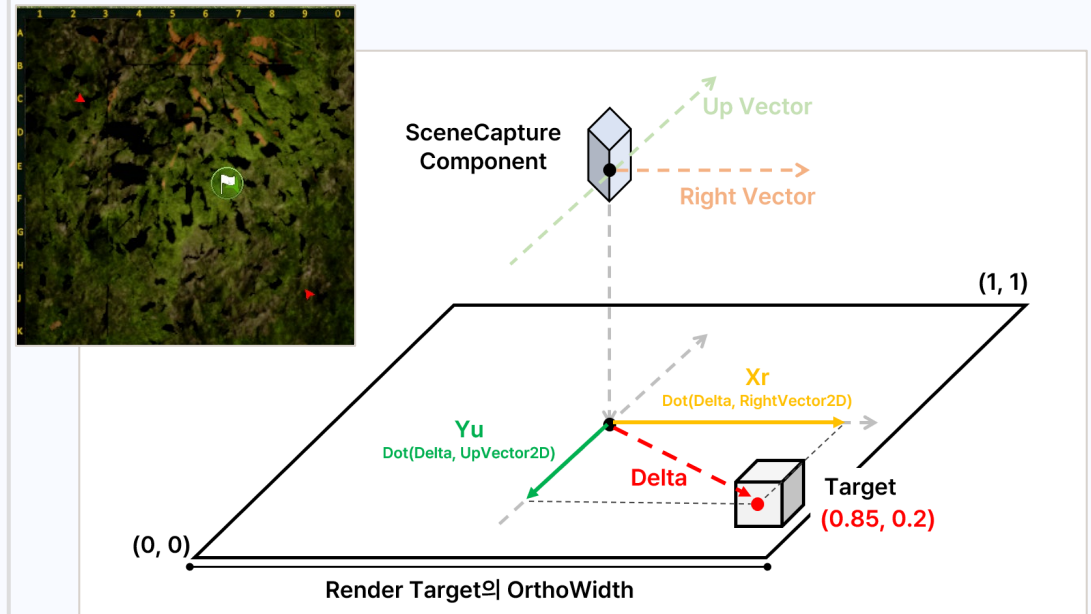
카메라 기준의 평면 벡터와 내적을 통해 정규화

2 정규화된 UV좌표 → 위젯의 픽셀 좌표

```
// 픽셀 좌표 변환
OutPixel = FVector2D(InX: UV.X * LayerSize.X, InY: (1.f - UV.Y) * LayerSize.Y);
```

UMG 좌표계와 월드 좌표계의 Y축 반전 문제를 보정

▶ 월드 좌표 → UV 좌표



Local Tanks

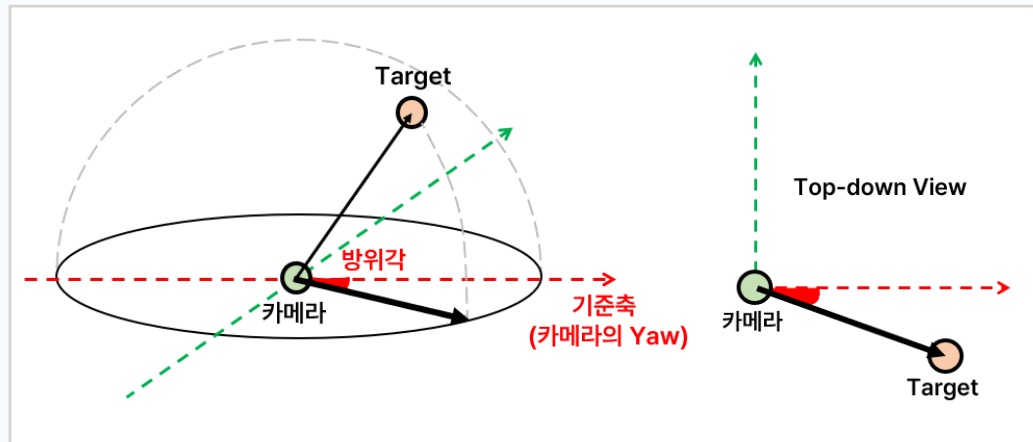
[Off Screen Indicator UI 소스코드](#)

점령지 Off-Screen Indicator UI

ProjectWorldToScreen()를 활용해 월드 좌표를 스크린으로 투영해 OnScreen여부를 판정합니다.
OffScreen이면 **카메라 기준의 점령지 방위각을 계산해 해당 방향으로 가장자리 교차점에** UI를 배치하고,
Atan2 로 각도로 변환해 UI의 화살표가 정확한 타겟 방향을 가리키도록 구현했습니다.

구현 방식

1 카메라 기준 방위각 → 2D 화면 방향 벡터 구하기



카메라를 기준으로 Target(점령지)의 방위각(ΔYaw)을 계산,
화면 중심에서 목표를 향하는 단위 방향 벡터를 생성

2 2D Ray + 교차점 구하기



화면 중심 → 해당 방향으로 Ray를 투사해 화면 경계 교차점을 산출,
Off-Screen Indicator의 위치와 각도를 지정

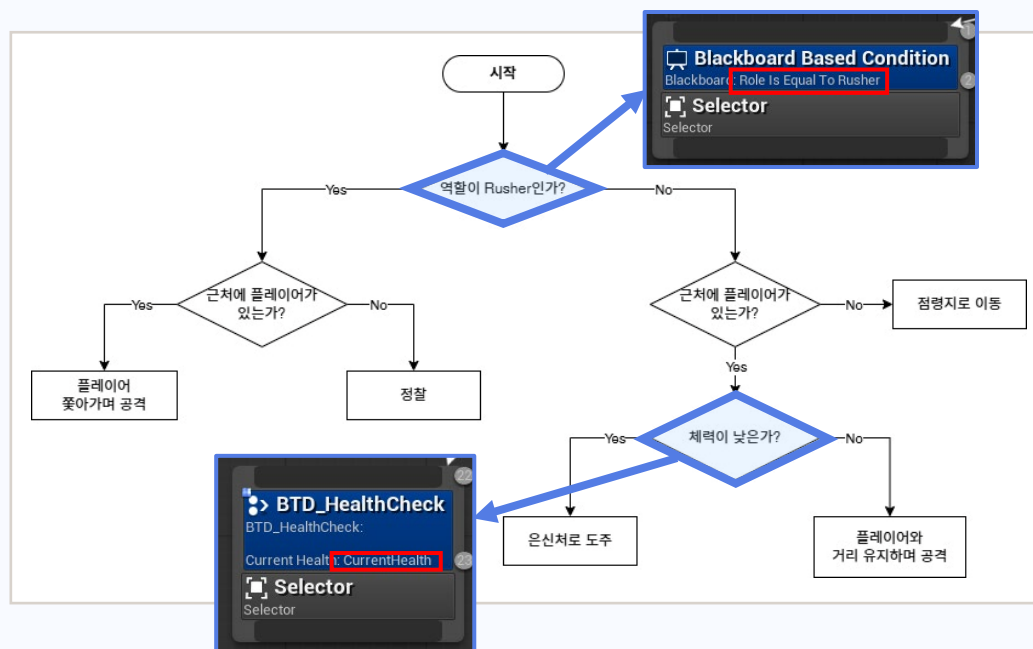
Local Tanks

BT를 활용한 AI 탱크 개발

언리얼엔진 Behavior Tree와 블루프린트를 활용해 AI 탱크의 전술 행동을 설계했습니다.

역할(Enum) 기반 전략 분기로 돌진형/은신형 AI를 하나의 시스템에서 구현하였으며,
AI Perception을 통한 **시야 기반 타겟팅**과 **체력 상태 기반 행동 전환**을 지원합니다.

▶ 행동 로직



▶ 탱크 이미지



Hider | 은신형 탱크

플레이어와
일정 거리 유지하며 공격



Rusher | 돌진형 탱크

점령지 근처에서 정찰
및 점령지로 이동

Local Tanks

트러블 슈팅

[Off Screen Indicator UI 각도 계산 소스코드](#)

▶ Off-Screen Indicator | 화살표 회전 각도 왜곡

상황

뷰포트와 UI Canvas의 비율이 달라 중심→타겟 벡터가 Canvas에서 왜곡 되어 Indicator의 방향 화살표가 화면 밖 목표물을 정확히 가리키지 못하고 **덜 기울어진 각도로 표시되는 문제**가 있었습니다. 특히 대각선 방향에서 회전 각도가 45°가 되어야 할 상황에서도 30~35°처럼 왜곡되어 시각적 혼란을 주었습니다.

해결 방법

이를 해결하기 위해 **각도 계산을 월드/카메라 기준으로 변경**하였습니다. 카메라의 회전값 (Yaw)로 **시야각 단위 벡터를 생성**해 각도를 계산하였고 이로 인해 종횡비 영향을 제거해 일관된 회전 방향을 유지할 수 있게 되었습니다.

배운 점

UI좌표계와 3D월드 좌표계의 비율 차이가 실제 연산에 미치는 영향을 이해했습니다. UI/UX에서는 **종횡비 변화나 해상도 변화에서도 안정적인 계산**을 유지하는 설계가 중요함을 인식했습니다.



정확히 화면 모서리를 향해 있지 않고,
덜 기울어진 각도로 표시



위와 같은 방향에서 올바른 각도 표현

Local Tanks

트러블 슈팅

▶ Off-Screen Indicator | 해상도 변경 시 UI 위치 불일치

상황

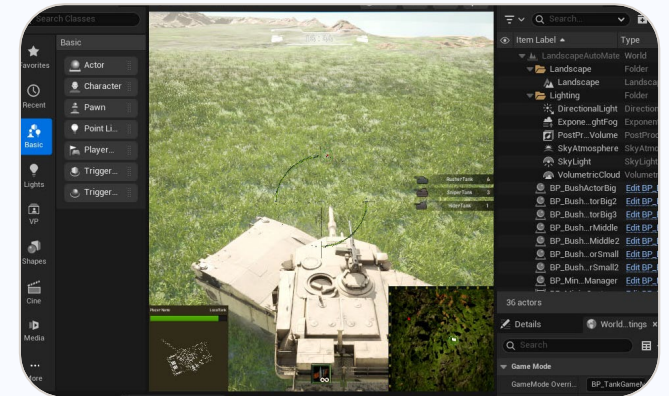
게임 실행 중 창 모드 전환 또는 해상도 변경이 발생하면 **Indicator UI**의 위치가 기존 위치에서 **틀어져 화면 밖으로 벗어나거나, 좌우 여백이 달라지는** 등의 문제가 발생했습니다. 이는 해상도를 변경해도 UI 위치가 초기 **Viewport** 기준으로 고정되어 있었기 때문입니다.

해결 방법

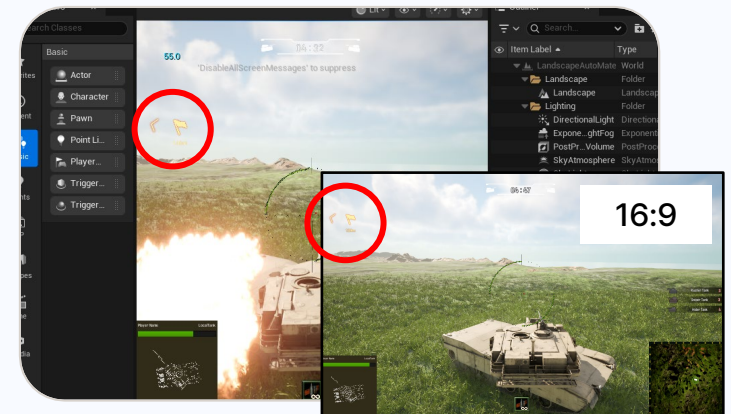
Viewport 크기가 동적으로 변경될 수 있다는 점을 고려하여 Tick에서 **Viewport 크기 변화 여부를 실시간으로 감지하고, 변화가 감지되면 Clamp 범위 및 화면 중앙 좌표 등을 재계산하도록 구조를 개선했습니다.**

배운 점

UI에서는 화면 크기나 비율이 언제든지 변경될 수 있다는 전제를 고려해야 하며, 정적인 초기값만을 기준으로 계산하는 것은 쉽게 틀어질 수 있다는 것을 경험했습니다. 특히 화면 가장자리와 관련된 UI요소는 **실시간 변화를 감지하는 시스템을 반드시 포함해야** 한다는 점을 배웠습니다.

[Off Screen Indicator UI 해상도 변화 소스코드](#)


해상도 변경시 UI가 뷰포트 밖으로 사라짐



해상도가 변해도 기존과 같은 위치 유지

04 Right 4 Dead 2

[시연영상](#)

[깃허브 링크](#)



▶ 프로젝트 개요

다양한 무기와 전술 스킬이 핵심인 FPS 게임의 **무기 시스템**을 설계하고 구현했습니다. **객체 지향 설계**를 통해 확장 가능하며 플레이어에게 몰입감 있는 전투 경험을 제공하고자 했습니다.

▶ 개발 진행 기간

2025.02 - 03

2025.08에 코드 개선

▶ 작업 인원

2인

▶ 담당 역할

- 무기 구조 설계, 무기 슬롯 시스템
- 무기 별 공격 로직
- 플레이어 / 플레이어 관련 전투 UI

▶ 사용한 기술

Unreal Engine 5, C++

Right 4 Dead 2

[USlotComponent 소스코드](#)

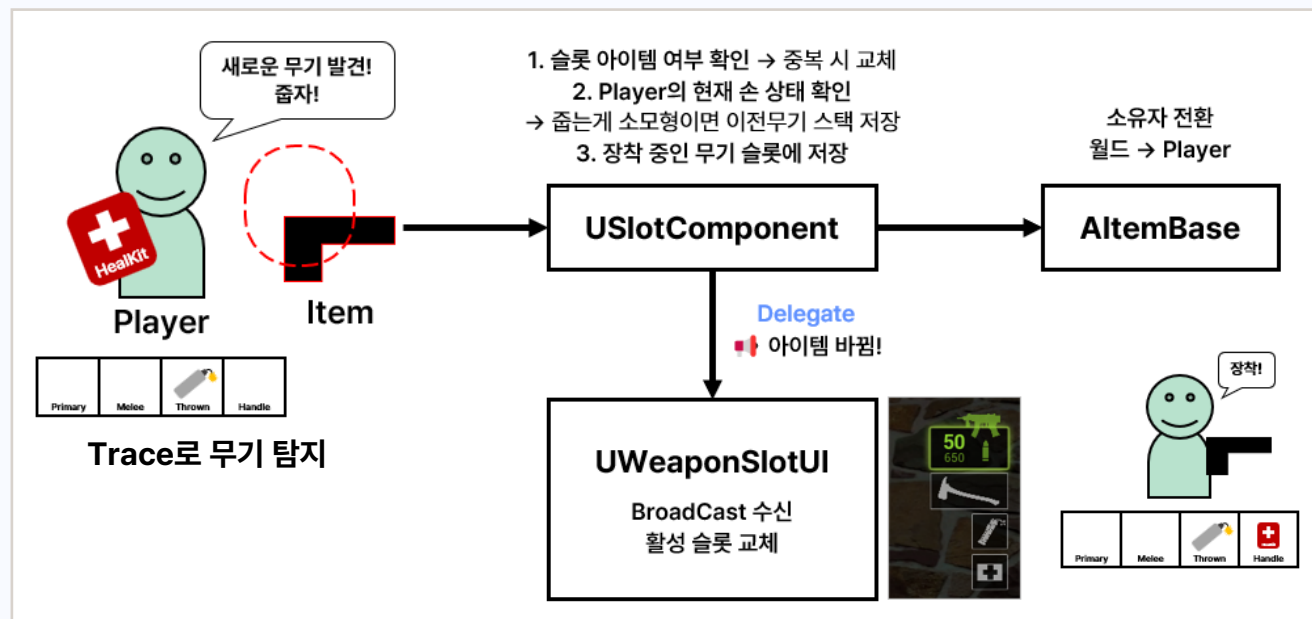
무기 슬롯 시스템

모든 아이템 흐름을 관리하는 **USlotComponent**를 중심으로

무기 탐지 → 줍기 → 장착 → 해제/버리기 → 무기 전환 → UI 업데이트까지의 전체 루프를 하나의 일관된 시스템으로 구현했습니다.

또한 UI, Input, 아이템을 **델리게이트** 기반으로 연결해 항상 최신 상태를 반영합니다.

▶ 슬롯 장착 다이어그램



▶ 슬롯 아이템



Right 4 Dead 2

[밀쳐내기 소스코드](#)

무기 별 공격 로직

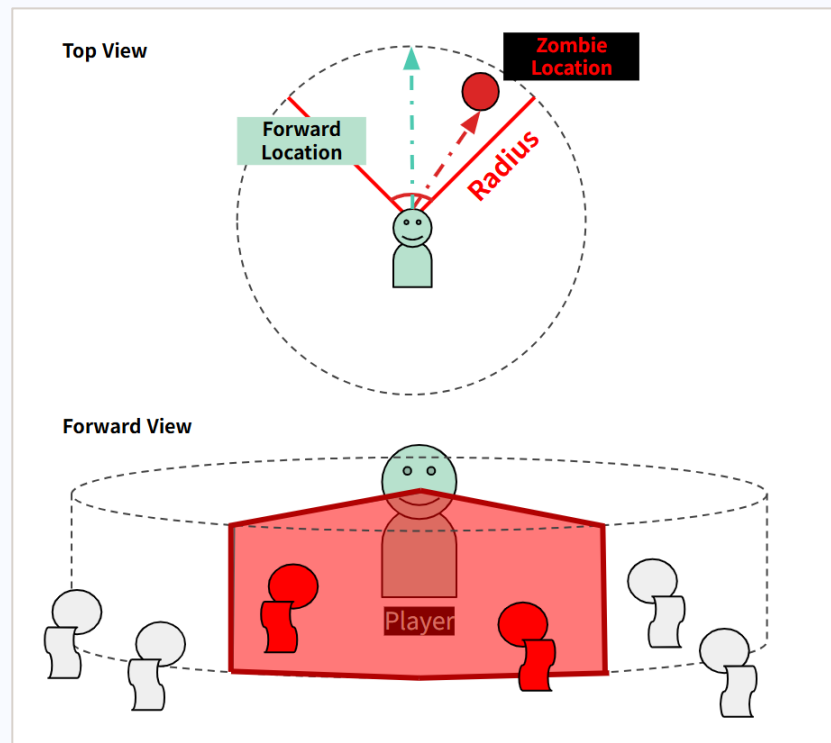
근접·투척·총기 등 다양한 무기 유형별 공격 로직과 **밀쳐내기** 로직을 설계했습니다.

1 밀쳐내기

```
//조건1: 좀비가 맞았는지
if (ACommonZombie* CommonZombie = Cast<ACommonZombie>(OtherActor))
{
    //조건2: 좀비가 플레이어 전방 기준 좌우 45도 안에 있는가?
    FVector ZombieLocation = (CommonZombie->GetActorLocation() - GetActorLocation()).GetSafeNormal();
    FVector SurvivorForwardVector = GetActorForwardVector();

    //내적으로 각도 구하기
    float Angle = FMath::RadiansToDegrees( RadVal: FMath::Acos( Value: FVector::DotProduct( A: SurvivorForwardVector, B: ZombieLocation)))
    if (Angle <= 45.0f)
    {
        //피해 적용
        UGameplayStatics::ApplyDamage( DamagedActor: CommonZombie, BaseDamage: 10, EventInstigator: Char->GetController(), DamageCauser: this,
    }
}
```

1. AnimNotify를 이용해 몽타주가 실행될 때 **Cylinder Mesh**를 생성
2. Cylinder Mesh와 **Overlap 액터**를 판단해
 - 2-1. 좀비가 맞았는지?
 - 2-2. 맞은 좀비가 플레이어 전방 기준 좌우 45도 이내에 있는지?
3. **내적으로 각도를 계산**한 후 해당 좀비에게 피해 적용



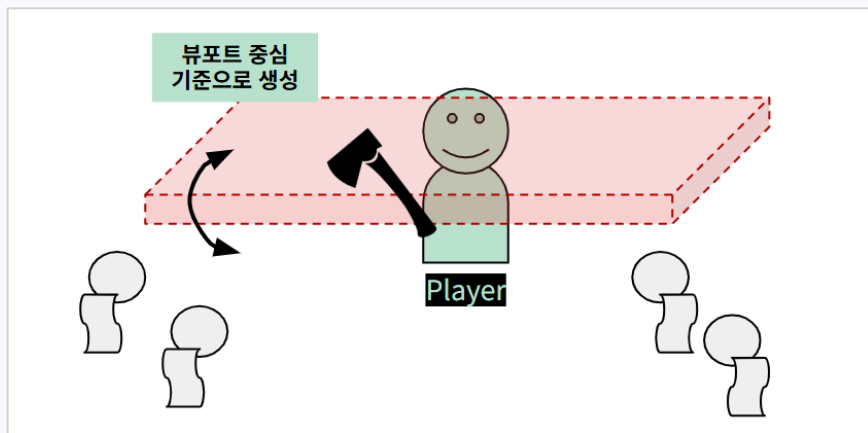
Right 4 Dead 2

[근접무기 소스코드](#)

무기 별 공격 로직

근접·투척·총기 등 다양한 **무기 유형별 공격 로직**과 **밀쳐내기** 로직을 설계했습니다.

2 근접 무기



플레이어 뷰포트를 중심으로 박스 형태의 Sweep을 수행

▶ 다중 부위 피격 처리

한 좀비의 여러 부위가 Hit되었을 때, 좀비를 키(Key)로 맞은 부위들을 값(Value)으로 저장하는 **TMap** 선언

```
// 한 액터(좀비)의 여러 부위가 박스 영역 안에 들어왔을 때 분류
// 한 번만 타격해야됨!
TMap<AActor*, TArray<FName>> HitMap;

TArray<AActor*> Actors;
HitMap.GetKeys([&] Actors);

for (auto Actor : Actors)
{
    int HighPriority = INT_MAX;
    FName HighPriorityBoneName;

    for (auto BoneName : HitMap[Actor])
    {
        int Priority = BoneMap[BoneName];
        if (Priority < HighPriority)
        {
            HighPriority = Priority;
            HighPriorityBoneName = BoneName;
        }
    }
}
```

```
if (false == HitMap.Contains(Actor))
```

1. 처음 맞은 좀비라면,
새로운 TArray를 만들고
HitMap에 저장

```
else
```

2. 이미 맞은 좀비라면,
TArray에 맞은 부위 추가

3. TArray를 순회하며
우선순위 파악 후 공격

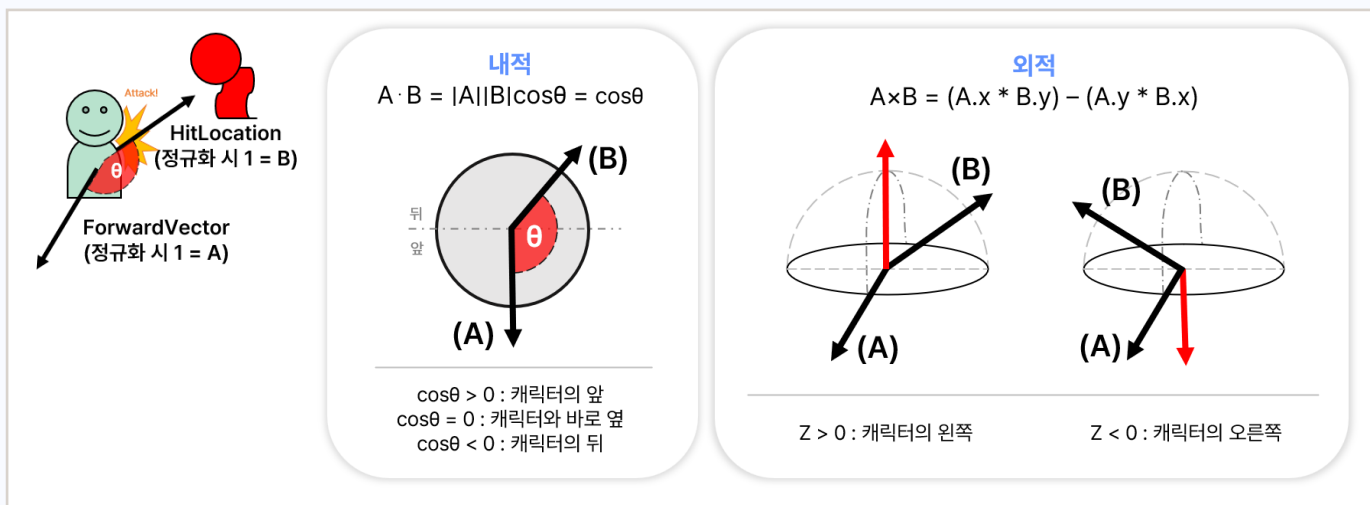
Right 4 Dead 2

[피해방향 Indicator UI 소스코드](#)

피해 방향 Indicator UI

내적과 외적을 이용해 플레이어가 어느 방향에서 공격받고 있는지를 나타내는 Indicator UI를 구현했습니다.

▶ 내적과 외적을 이용한 방향 판단



```
// cos, sin 계산
const float CosTheta = FVector::DotProduct( A: Fwd, B: DirWorld);
const float SinTheta = FVector::CrossProduct( A: Fwd, B: DirWorld).Z; // 우-hand 기준, 좌우 방향 부호

// 각도 계산 (왼쪽: +, 오른쪽: - 가 됨)
const float AngleDeg = FMath::RadiansToDegrees( RadVal: FMath::Atan2( Y: SinTheta, X: -CosTheta));
```

▶ Indicator UI



좀비가 플레이어 앞, 좌측에 있을 때



좀비가 플레이어 뒤, 좌측에 있을 때

Right 4 Dead 2

트러블 슈팅

▶ 무기 슬롯 중복 저장 버그

상황

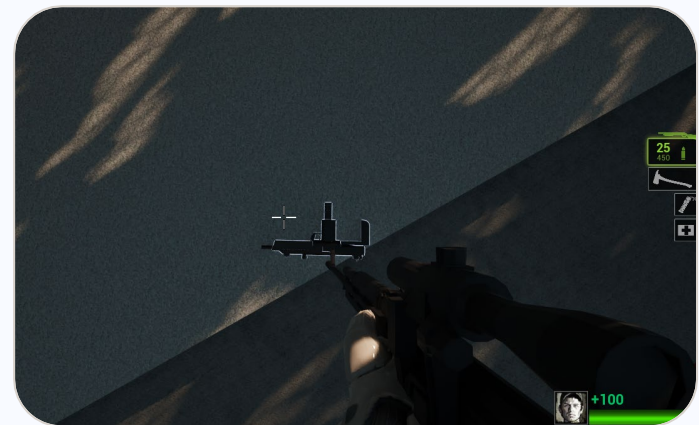
동일 슬롯의 무기를 장착하면 두 무기가 교체되지 않고 동시에 슬롯에 저장되는 버그가 발생했습니다. 그 결과, 무기 전환 키(예: 1번 키)를 반복해서 누를 때 두 무기가 번갈아 장착되는 현상이 나타나 플레이에 혼란을 주었습니다.

해결 방법

기존 TMap 기반 인벤토리 구조에서 **InHands** 무기와 슬롯 간 상태 동기화가 보장되지 않는 것이 원인이었습니다. 이를 해결하기 위해 **USlotComponent**를 도입해 슬롯 관리와 장착 과정을 단일 책임으로 통합하고, **줍기 → 슬롯 배치/대체 → In-Hands 장착 → UI 갱신을 원자적 순서로 처리했습니다**. 또한 동일 슬롯에 새 무기를 줍는 경우 **Swap** 과정을 명시적으로 수행해 중복 참조를 방지했습니다.

배운 점

단순히 코드 수정보다 **상태 관리의 책임 주체를 명확히 하는 구조적 개선**이 문제 해결에 효과적이라는 것을 배웠으며, 아이템 상태, 입력, UI를 아우르는 **단일 책임 원칙 기반 설계의 중요성**을 알게되었습니다.



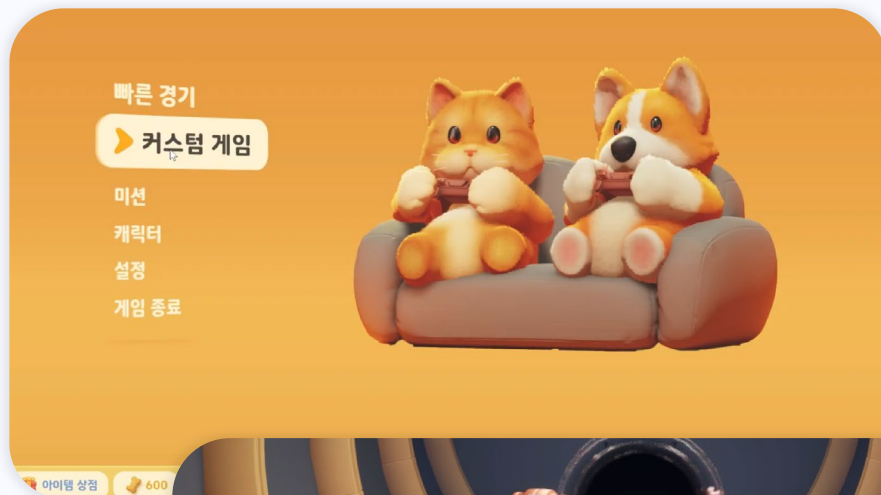
동일 슬롯 무기 교체 시 Drop, UI 업데이트

05 Party Banimals

[시연영상](#)

[깃허브 링크](#)

멀티플레이



▶ 프로젝트 개요

물리 기반 상호작용이 핵심인 <블랙홀> 맵을 개발, 이를 배경으로 하는 멀티플레이어 생존 게임을 구현했습니다. 네트워크 환경에서 플레이어 간 안정적인 물리 동기화와 전략적 협동/경쟁 플레이를 구현하는 데 집중했습니다.

▶ 개발 진행 기간

2025.03 - 04

▶ 작업 인원

3인

▶ 담당 역할

- Steam 연동 로비
- <블랙홀 연구실> 맵, 맵 게임 시스템
- 중력 컴포넌트
- 관전자 모드 구현

▶ 사용한 기술

Unreal Engine 5, C++

Party Banimals

[관련 소스코드](#)
[게임 시스템 소스코드](#)

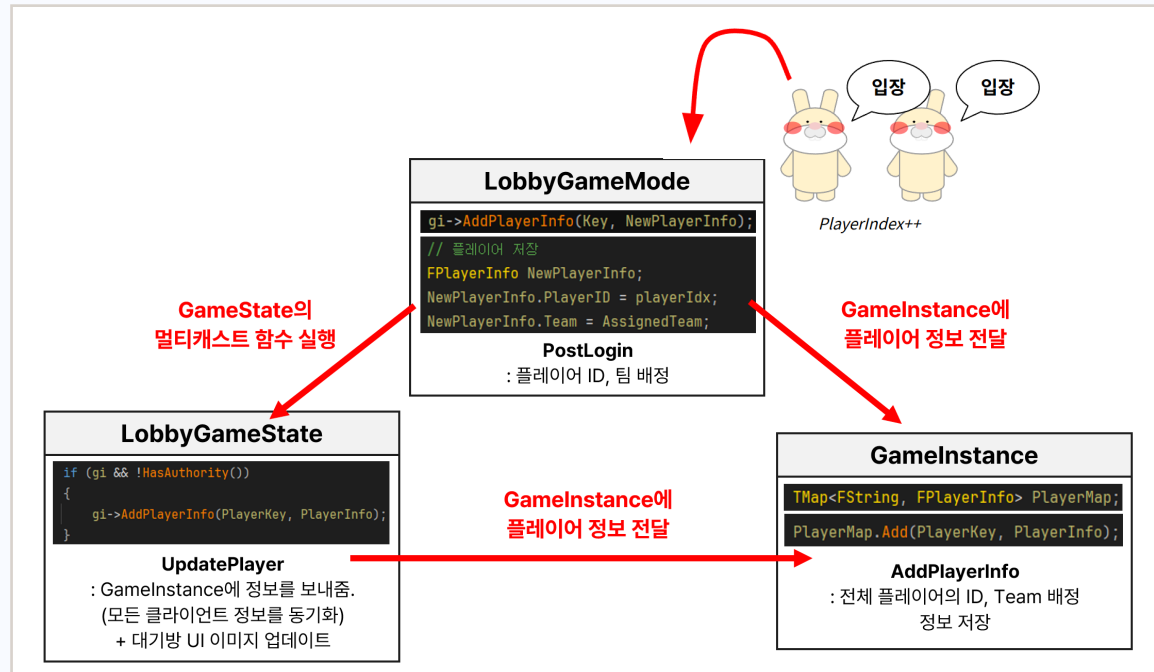
게임 시스템

멀티플레이 팀 배정 및 게임 상태 동기화

플레이어 접속 시 **GameMode**에서 식별정보(**FPlayerInfo**)를 생성하고, 현재 맵 설정에 따라 자동으로 팀을 배정합니다. 생성된 정보는 **GameInstance**에 저장되어 레벨 전환 후에도 유지됩니다.

또한 **GameState**는 실시간으로 생존 여부를 판단하여 최종 승리 팀을 서버에서 결정해 동기화합니다.

▶ 구조 다이어그램



▶ 팀 배정



러기지, 4인 1팀

블랙홀, 2인 1팀

```

// 들어온 순서대로
teamCnt = (playerIdx % teamMaxPlayers) + 1; // 레드(1), 블루(2), 옐로(3), 그린(4)
ETeamType AssignedTeam = static_cast<ETeamType>(teamCnt);

// 플레이어 저장
FPlayerInfo NewPlayerInfo;
NewPlayerInfo.PlayerID = playerId;
NewPlayerInfo.Team = AssignedTeam;
  
```

Party Banimals

[UGravityComponent 소스코드](#)

Gravity Component

UGravityComponent를 통해 블랙홀 중심 방향의 힘을 계산하여 **중력 및 궤도 운동을 서버에서 처리**하고 클라이언트는 위치와 속도만 수신하는 구조로 설계하였습니다.

블랙홀 연구실 맵은 총 4페이지로, 반경과 속도가 변화하며 난이도가 점차 상승합니다.

구현 방식

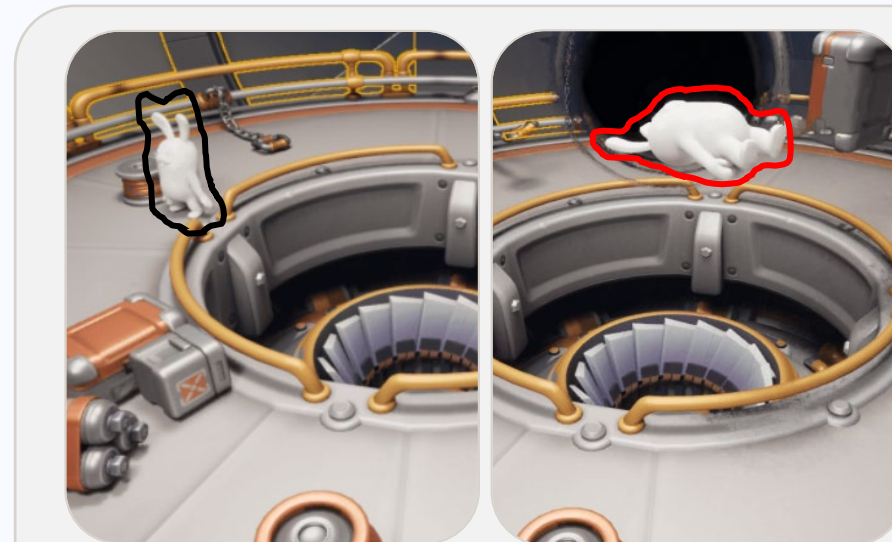
1 FinalVelocity 계산 (중력 + 궤도운동)

```
FVector GravityVelocity = GravityDir * GravityScale;  
FVector OrbitVelocity = (TargetPosition - ObjectLocation) * InterpSpeed;  
FVector FinalVelocity = GravityVelocity + OrbitVelocity;
```

2 Multicast로 클라이언트 동기화

```
if (!GetOwner()->HasAuthority()) // 서버가 아니라면  
{  
    if (PlayerCharacter)  
    {  
        // 서버에서 받은 속도를 그대로 적용  
        PlayerCharacter->GetCapsuleComponent()->SetPhysicsLinearVelocity(OrbitVelocity, bAddToC  
        PlayerCharacter->GetCapsuleComponent()->SetPhysicsAngularVelocityInDegrees(AngularVeloci  
    }  
}
```

블랙홀 이미지



Gravity Component가 부착된 액터만 블랙홀 중력에 반응

▶ 컴포넌트로 설계해 다양한 액터에 **재사용 가능**하며, 멀티플레이 환경에서도 **안정적인 궤도 운동**이 가능합니다.

Party Banimals

관전자 모드

플레이어 사망 시 게임을 즉시 종료하지 않고 **관전자 전용 Pawn으로 전환**해 경기를 관전합니다.
관전자가 된 플레이어는 코인을 소비해 **물리 충돌 아이템**을 맵에 투척해 **경기에 간접적으로 참여**할 수 있습니다.

구현 방식

1 서버에서 사망 처리 → 관전자 전환

2 관전자 Pawn 생성

```
ABlackholeSpectator* Spectator = GetWorld()->SpawnActorDeferred<ABlackholeSpectator>

if (Spectator)
{
    UGameplayStatics::FinishSpawningActor(Spectator, SpawnTransform: Target->GetActorTransform());
    PlayerController->Possess(Spectator); 관전자 Pawn을 Possess

    // 기존 플레이어 인풋 제거하자
    if (UEnhancedInputLocalPlayerSubsystem* inputsys = ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(GetWorld()))
    {
        inputsys->ClearAllMappings(); 기존 플레이어의 Input 제거
        inputsys->AddMappingContext(IMC_Spectator, Priority: 0);
    }

    pc = Cast<APlayerController>(Src: GetController());

    PlayerInputComponent->ClearActionBindings();
    PlayerInputComponent->ClearAxisBindings();
    PlayerInputComponent->AxisBindings.Empty();
}
```

3 관전자 상호작용(투척)

1. ClientSpectator 에서 화면 클릭

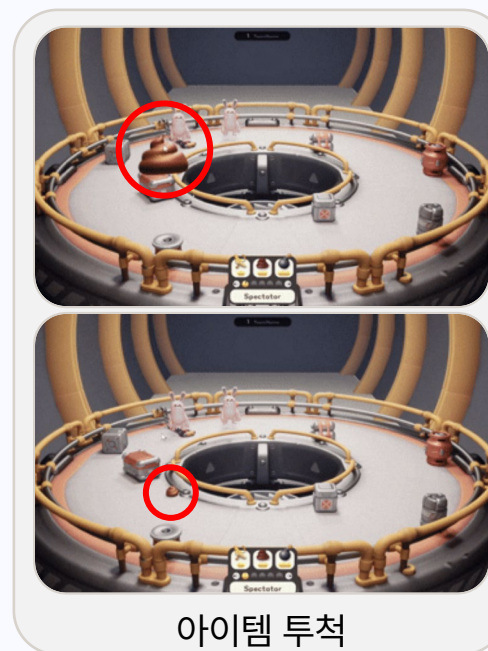
```
PC->DeprojectMousePositionToWorld() → 좌표 변환  
GetWorld()->LineTraceSingleByObjectType → Trace
```

ServerRPC

SpawnLocation
TargetLocation

2. Server에서 Multicast

모든 플레이어 화면에서
동일한 위치에
투척 아이템 생성



Party Banimals

트러블 슈팅

▶ 궤도 운동 멀티플레이 동기화 문제

상황

블랙홀 궤도 운동을 구현할 때 SetActorLocation을 이용해 **Tick마다 접선 방향 이동하고 중심과의 거리를 보정**하는 방식으로 처리했습니다. 하지만 이 계산은 각 클라이언트 로컬로 실행되면서 멀티플레이 환경에서는 **Tick 오차 및 네트워크 지연** 등의 영향으로 위치 불일치 또는 순간이동처럼 보이는 현상이 나타났습니다.

해결 방법

위치 자체를 직접 갱신하는 대신, **서버에서 궤도 속도를 물리적으로 계산**하고

SetPhysicsLinearVelocity로 적용해 물리 기반 이동으로 전환했습니다.

계산된 속도는 Multicast로 클라이언트에 전파되어 모든 플레이어가 동일한 속도로 공전하도록 동기화할 수 있었습니다.

배운 점

멀티플레이 환경에서는 Tick 기반 위치 보정보다 서버 주도 물리 기반 이동이 더 안정적입니다. 또한 **SetPhysicsLinearVelocity의 엔진 내부 보간 기능**과 함께 사용할 때 동기화 품질을 크게 향상시킬 수 있다는 점을 배웠습니다.



```
// 새 위치로 이동하자  
GetOwner()->SetActorLocation(SmoothedPosition);
```

```
// 부드러운 이동  
PlayerCharacter->GetCapsuleComponent()->SetPhysicsLinearVelocity(NewVec(TargetPosition - ObjectPosition));  
// 회전 속도 적용  
PlayerCharacter->GetCapsuleComponent()->SetPhysicsAngularVelocityInDegrees(AngularVelocity);
```


End of Document

감사합니다!

CONTACT

zlagksnf@gmail.com

010-4741-8904